

Techniques de compilation

Auditoire : CPI-2

Année Universitaire : 2025-2026

Code Classroom: **6xqrgwgl**

Dr.-Ing. Rakia Saidi

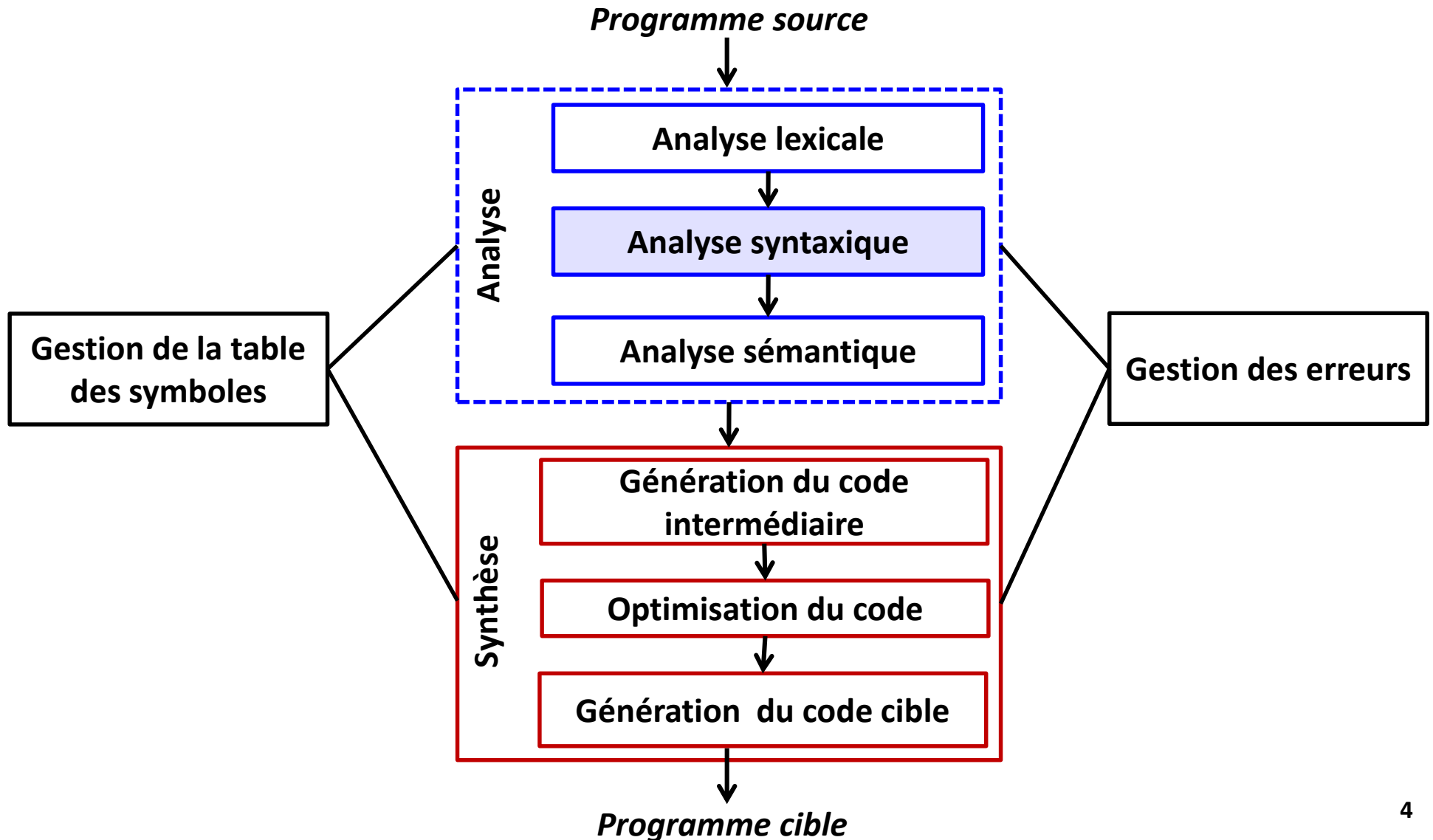
Maitre assistante en informatique
rakia.saidi@isimg.tn

Plan de ce cours

- **Chapitre 1**: Introduction à la compilation
- **Chapitre 2**: Rappels à la théorie des langages
- **Chapitre 3**: Analyse lexicale
- **Chapitre 4**: Analyse syntaxique
- **Chapitre 5**: Analyse sémantique
- **Chapitre 6**: Génération de code

Chapitre 4 :Analyse syntaxique

1. Introduction



1. Introduction

Principe

- Comme l'analyseur lexical, la construction d'un analyseur syntaxique est basée sur un ensemble d'outils théoriques.
 - Tout langage de programmation possède des règles qui indiquent la structure syntaxique d'un programme bien formé
 - La syntaxe d'un langage peut être décrite par une grammaire
 - Les grammaires sont des outils pour décrire les langages, inspirés des grammaires utilisées pour les phrases du langage naturel.
- L'analyseur syntaxique d'un compilateur est basé sur les grammaires non contextuelles ou hors contexte

2. Mise en œuvre d'un analyseur syntaxique

- Deux familles d'analyseur syntaxique
 - L'analyse ascendante:
 - Un analyseur syntaxique droit construit l'arbre de dérivation à partir de ses feuilles et en effectuant des dérivations les plus à droite. C'est la famille des analyseurs LR (Left scanning, Rightmost derivation).
 - L'analyse descendante:
 - Un analyseur syntaxique gauche construit l'arbre de dérivation à partir de sa racine et en effectuant des dérivations les plus à gauche. C'est la famille des analyseurs LL (Left scanning, Leftmost derivation).
 - Les méthodes non déterministes
 - Analyse syntaxique prédictive ou parallèle
 - Analyse syntaxique avec retour en arrière
 - Les méthodes déterministes

2. Mise en œuvre d'un analyseur syntaxique

Analyse ascendante

Principe

- Construire l'arbre de dérivation en démarrant des unités lexicales (feuilles) d'une chaîne jusqu'à arriver à la racine (axiome de départ)
- Des réductions successives jusqu'à retrouver l'axiome de départ de la grammaire
- On procède à la réduction des symboles en allant de gauche à droite.

2. Mise en œuvre d'un analyseur syntaxique

Analyse descendante

Analyse descendante déterministe

- Construire une table qui nous dit:
 - « *Quand on lit tel caractère et qu'on en est à dériver tel symbole non-terminal, alors on applique telle règle* »
- Cette table s'appelle table d'analyse

2. Mise en œuvre d'un analyseur syntaxique

Analyse descendante déterministe

Table d'analyse LL

- Pour construire cette table d'analyse, on a besoin de définir deux ensembles:
 - L'ensemble des PREMIER
 - L'ensemble des SUIVANT

2. Mise en œuvre d'un analyseur syntaxique

Analyse descendante déterministe

Table d'analyse LL - Calcul de PREMIER

- Pour toute chaîne α composée de symboles terminaux et non terminaux, on cherche $\text{PREMIER}(\alpha)$:
 - L'ensemble de tous les terminaux qui peuvent commencer une chaîne qui se dérive de α .
 - L'ensemble de toutes les lettres a telle qu'il existe une dérivation $\alpha \xrightarrow{*} a\beta$, β une chaîne composée de symboles terminaux et de non-terminaux.
 - **$\text{PREMIER}(\alpha) = \{a \in V_T / \exists \beta \in (V_T \cup V_N)^* \text{ et } \alpha \xrightarrow{*} a\beta\} \cup \{\epsilon \text{ si } \alpha \xrightarrow{*} \epsilon\}$**

2. Mise en œuvre d'un analyseur syntaxique

Analyse descendante déterministe

Table d'analyse LL - Calcul de PREMIER

- Algorithme de construction des ensembles des PREMIER

1. Si X est non-terminal et $X \rightarrow Y_1, Y_2, \dots, Y_n$ est une production de la grammaire (avec Y_i symbole non-terminal ou terminal) alors
 - Ajouter les éléments de $\text{PREMIER}(Y_1)$ sauf ϵ dans $\text{PREMIER}(X)$
 - S'il existe j ($j \in \{2, \dots, n\}$) tel que pour tout $i=1, \dots, j-1$ on a ϵ appartient à $\text{PREMIER}(Y_i)$ alors ajouter les éléments de $\text{PREMIER}(Y_j)$ sauf ϵ dans $\text{PREMIER}(X)$
 - Si pour tout $i=1, \dots, n$ on a ϵ appartient à $\text{PREMIER}(Y_i)$ alors ajouter ϵ dans $\text{PREMIER}(X)$
2. Si X est un non-terminal et $X \rightarrow \epsilon$ une production alors ajouter ϵ dans $\text{PREMIER}(X)$
3. Si X est un terminal alors $\text{PREMIER}(X) = \{X\}$
4. Recommencer jusqu'à on a rien de nouveau à ajouter dans les ensembles PREMIER.

2. Mise en œuvre d'un analyseur syntaxique

Analyse descendante déterministe

Table d'analyse LL - Calcul de PREMIER

- Calculer les ensembles PREMIER des symboles non-terminaux pour les grammaires suivantes:

- Exemple 1:

$V_T = \{a, b, c, d\}$, $V_N = \{S, B, P\}$, Axiome = S , $P =$

$$\left\{ \begin{array}{l} S \rightarrow Ba \\ B \rightarrow cP \mid bP \mid P \mid \varepsilon \\ P \rightarrow dS \end{array} \right.$$

$\text{PREMIER}(P) = \{d\}$

$\text{PREMIER}(B) = \{c, b, d, \varepsilon\}$

$B \rightarrow cP$ donc $c \in \text{PREMIER}(B)$

$B \rightarrow bP$ donc $b \in \text{PREMIER}(B)$

$B \rightarrow P$ donc $\text{PREMIER}(P) \in \text{PREMIER}(B)$

$B \rightarrow \varepsilon$ donc $\varepsilon \in \text{PREMIER}(B)$

$\text{PREMIER}(S) = \text{PREMIER}(B) = \{c, b, d, \varepsilon\}$

2. Mise en œuvre d'un analyseur syntaxique

Analyse descendante déterministe

Table d'analyse LL - Calcul de PREMIER

- Exemple 2:

$$V_T = \{a, b, c, d\}, V_N = \{S, B, P\}, \text{Axiome} = S, P = \begin{cases} S \rightarrow Sc | Ba \\ B \rightarrow BP a | bP b | P | \varepsilon \\ P \rightarrow dS \end{cases}$$

$$\text{PREMIER}(P) = \{d\}$$

$$\text{PREMIER}(B) = \{b, d, \varepsilon\}$$

$$B \rightarrow bPb \text{ donc } b \in \text{PREMIER}(B)$$

$$B \rightarrow P \text{ donc } \text{PREMIER}(P) \in \text{PREMIER}(B)$$

$$B \rightarrow \varepsilon \text{ donc } \varepsilon \in \text{PREMIER}(B)$$

$$\text{PREMIER}(S) = \{\text{PREMIER}(B) \text{ sauf } \varepsilon\} \cup \{a\} = \{b, d, a\}$$

2. Mise en œuvre d'un analyseur syntaxique

Analyse descendante déterministe

Table d'analyse LL - Calcul de PREMIER

- Exemple 3:

$V_T = \{+, -, *, /, nb, (,)\}$, $V_N = \{E, E', T, T', F\}$, Axiome = E, P =

$$\left\{ \begin{array}{l} E \rightarrow TE' \\ E' \rightarrow +TE' \mid -TE' \mid \varepsilon \\ T \rightarrow FT' \\ T' \rightarrow *FT' \mid /FT' \mid \varepsilon \\ F \rightarrow (E) \mid nb \end{array} \right.$$

$\text{PREMIER}(F) = \{(, nb\}$

$\text{PREMIER}(T) = \text{PREMIER}(F) = \{(, nb\}$

$\text{PREMIER}(T') = \{*, /, \varepsilon\}$

$\text{PREMIER}(E') = \{+, -, \varepsilon\}$

$\text{PREMIER}(E) = \text{PREMIER}(T) = \{(, nb\}$

2. Mise en œuvre d'un analyseur syntaxique

Analyse descendante déterministe

Table d'analyse LL - Calcul de SUIVANT

- Pour tout symbole non-terminal A, on cherche SUIVANT(A), c'est:
 - L'ensemble de tous les symboles terminaux qui peuvent apparaître immédiatement à droite de A dans une dérivation.
 - $\text{SUIVANT}(A) = \{a \in V_T / \exists \alpha, \beta \in (V_T \cup V_N)^* \text{ où } A \xrightarrow{*} \alpha A a \beta\}$

2. Mise en œuvre d'un analyseur syntaxique

Analyse descendante déterministe

Table d'analyse LL - Calcul de SUIVANT

- Algorithme de construction des ensembles des SUIVANT

1. Ajouter un marqueur de fin de chaîne (\$) par exemple) à $SUIVANT(S)$, (S étant l'axiome de la grammaire)
2. Pour chaque production $A \rightarrow \alpha B \beta$, où B est un non-terminal, alors ajouter le contenu de $PREMIER(\beta)$ à $SUIVANT(B)$ sauf ϵ .
3. Pour chaque production $A \rightarrow \alpha B$ alors ajouter $SUIVANT(A)$ à $SUIVANT(B)$
4. Pour chaque production $A \rightarrow \alpha B \beta$ avec $\epsilon \in PREMIER(\beta)$ ajouter $SUIVANT(A)$ à $SUIVANT(B)$
5. Recommencer à partir de l'étape 3 jusqu'à ce qu'on ajoute rien de nouveau dans les ensembles SUIVANT

2. Mise en œuvre d'un analyseur syntaxique

Analyse descendante déterministe

Table d'analyse LL - Calcul de SUIVANT

- Calculer les ensembles SUIVANT des symboles non-terminaux pour les grammaires suivantes:

• Exemple 1:

$$V_T = \{a, b, c, d\}, V_N = \{S, B, P\}, \text{Axiome} = S, P = \begin{cases} S \rightarrow Ba \\ B \rightarrow cP \mid bP \mid P \mid \epsilon \\ P \rightarrow dS \end{cases}$$

Appliquer règle 2 ($A \rightarrow \alpha B \beta$) : ajouter $\text{PREMIER}(\beta)$ sauf ϵ à $\text{SUIVANT}(B)$:

$S \rightarrow B \underline{a}$ avec $A=S, B=B, \alpha=\epsilon$ et $\beta=a$ donc $a \in \text{SUIVANT}(B)$

Appliquer règle 3 ($A \rightarrow \alpha B$) : ajouter $\text{SUIVANT}(A)$ à $\text{SUIVANT}(B)$:

$B \rightarrow cP$ ($A=B, B=P, \alpha=c$)

$B \rightarrow bP$ ($A=B, B=P, \alpha=b$)

$B \rightarrow P$ ($A=B, B=P, \alpha=\epsilon$)

} donc ajouter $\text{SUIVANT}(B)$ à $\text{SUIVANT}(P)$

$P \rightarrow dS$ ($A=P, B=S, \alpha=d$) donc ajouter $\text{SUIVANT}(P)$ à $\text{SUIVANT}(S)$

	SUIVANT	PREMIER
S	\$, a	ϵ, b, c, d
B	a	b, c, d, ϵ
P	a	d

2. Mise en œuvre d'un analyseur syntaxique

Analyse descendante déterministe

Table d'analyse LL - Calcul de SUIVANT

- Exemple 2:
 $V_T = \{a, b, c, d\}$, $V_N = \{S, B, P\}$, Axiome = S, $P = \begin{cases} S \rightarrow Sc|Ba \\ B \rightarrow BP a|bPb|P|\varepsilon \\ P \rightarrow dS \end{cases}$

Appliquer règle 2 ($A \rightarrow \alpha B \beta$) : ajouter $\text{PREMIER}(\beta)$ sauf ε à $\text{SUIVANT}(B)$:

$S \rightarrow B \underline{a}$ avec $A=S$, $B=B$, $\alpha=\varepsilon$ et $\beta=a$ donc $a \in \text{SUIVANT}(B)$

$B \rightarrow bP \underline{b}$ avec $A=B$, $B=P$, $\alpha=b$ et $\beta=b$ donc $b \in \text{SUIVANT}(P)$

$B \rightarrow BP \underline{a}$ avec $A=B$, $B=P$, $\alpha=B$ et $\beta=a$ donc $a \in \text{SUIVANT}(P)$

$S \rightarrow S \underline{c}$ avec $A=A$, $B=S$, $\alpha=\varepsilon$ et $\beta=c$ donc $c \in \text{SUIVANT}(S)$

$B \rightarrow B \underline{P} a$ avec $B=B$, $\alpha=\varepsilon$ et $\beta=Pa$ donc $\text{PREMIER}(Pa) \setminus \{\varepsilon\} \in \text{SUIVANT}(B)$

Appliquer règle 3 ($A \rightarrow \alpha B$) : ajouter $\text{SUIVANT}(A)$ à $\text{SUIVANT}(B)$:

$B \rightarrow P$ ($A=B$, $B=P$, $\alpha=\varepsilon$) donc ajouter $\text{SUIVANT}(B)$ à $\text{SUIVANT}(P)$

$P \rightarrow dS$ ($A=P$, $B=S$, $\alpha=d$) donc ajouter $\text{SUIVANT}(P)$ à $\text{SUIVANT}(S)$

	SUIVANT	PREMIER
S	\$, c , a , b , d	b , d , a
B	a , d	b , d , ε
P	b , a , d	d

2. Mise en œuvre d'un analyseur syntaxique

Analyse descendante déterministe

Table d'analyse LL - Calcul de SUIVANT

- Exemple 3:

$V_T = \{+, -, *, /, nb, (,)\}$, $V_N = \{E, E', T, T', F\}$, Axiome = E , $P =$

$$\left\{ \begin{array}{l} E \rightarrow TE' \\ E' \rightarrow +TE' \mid -TE' \mid \varepsilon \\ T \rightarrow FT' \\ T' \rightarrow *FT' \mid /FT' \mid \varepsilon \\ F \rightarrow (E) \mid nb \end{array} \right.$$

$SUIVANT(E) = \{\$,)\}$

$SUIVANT(E') = \{\$,)\}$

$SUIVANT(T) = \{+, -,), \$\}$

$SUIVANT(T') = \{+, -,), \$\}$

$SUIVANT(F) = \{+, -, *, /,), \$\}$

2. Mise en œuvre d'un analyseur syntaxique

Analyse descendante déterministe

Table d'analyse LL - Calcul de SUIVANT

- Exemple 3:

Appliquer règle 2 ($A \rightarrow \alpha B \beta$) : ajouter $\text{PREMIER}(\beta)$ sauf ϵ à $\text{SUIVANT}(B)$:

$F \rightarrow (E)$ avec $A=F, B=E, \alpha=($ et $\beta=)$ donc $) \in \text{SUIVANT}(E)$

$E' \rightarrow +TE'$ ou $E' \rightarrow -TE'$ avec $B=T, \beta=E'$ donc $\text{PREMIER}(E') \setminus \{\epsilon\} \in \text{SUIVANT}(T)$

$T' \rightarrow *FT'$ ou $T' \rightarrow /FT'$ avec $B=F, \beta=T'$ donc $\text{PREMIER}(T') \setminus \{\epsilon\} \in \text{SUIVANT}(F)$

Appliquer règle 3 ($A \rightarrow \alpha B$) : ajouter $\text{SUIVANT}(A)$ à $\text{SUIVANT}(B)$:

$E \rightarrow TE'$ ($A=E, B=E', \alpha=T$) donc ajouter $\text{SUIVANT}(E)$ à $\text{SUIVANT}(E')$

$T \rightarrow FT'$ ($A=T, B=T', \alpha=F$) donc ajouter $\text{SUIVANT}(T)$ à $\text{SUIVANT}(T')$

Appliquer règle 4 ($A \rightarrow \alpha B \beta$) et $\epsilon \in \text{PREMIER}(\beta)$: ajouter $\text{SUIVANT}(A)$ à $\text{SUIVANT}(B)$

$E' \rightarrow +TE'$ ou $E' \rightarrow -TE'$ avec $A=E', B=T, \beta=E', \epsilon \in \text{PREMIER}(E')$: ajouter $\text{SUIVANT}(E')$ à $\text{SUIVANT}(T)$

$T' \rightarrow *FT'$ ou $T' \rightarrow /FT'$ avec $A=T', B=F, \beta=T', \epsilon \in \text{PREMIER}(T')$: ajouter $\text{SUIVANT}(T')$ à $\text{SUIVANT}(F)$

	SUIVANT	PREMIER
E	\$,)	(, nb
E'	\$,)	+, -, ϵ
T	+, - , \$,)	(, nb
T'	+, - , \$,)	*, /, ϵ
F	*, /, +, -, \$,)	(, nb

2. Mise en œuvre d'un analyseur syntaxique

Analyse descendante déterministe

Table d'analyse LL

- Une table d'analyse est un tableau M à deux dimensions qui indique pour chaque symbole non-terminal A et pour chaque symbole terminal a ou symbole $\$,$ la règle de production à appliquer.
 - Pour chaque règle de production $A \rightarrow \alpha$
 1. Pour tout $a \in \text{PREMIER}(\alpha)$ (et $a \neq \epsilon$), ajouter la production $A \rightarrow \alpha$ dans la case $M[A, a]$
 2. Si $\epsilon \in \text{PREMIER}(\alpha)$, alors pour chaque $b \in \text{SUIVANT}(A)$ ajouter la production $A \rightarrow \alpha$ dans la case $M[A, b]$
 - Chaque case vide est une erreur de syntaxe.

2. Mise en œuvre d'un analyseur syntaxique

Analyse descendante déterministe

Table d'analyse LL

- Construire la table d'analyse de l'exemple 3:

$$\begin{cases}
 E \rightarrow TE' \\
 E' \rightarrow +TE' \mid -TE' \mid \varepsilon \\
 T \rightarrow FT' \\
 T' \rightarrow *FT' \mid /FT' \mid \varepsilon \\
 F \rightarrow (E) \mid nb
 \end{cases}$$

$V_T = \{+, -, *, /, nb, (,)\}$, $V_N = \{E, E', T, T', F\}$, Axiome = E , $P =$

$V_N \backslash V_T$	nb	+	-	*	/	(	)	\$
E	$E \rightarrow TE'$					$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$	$E' \rightarrow -TE'$				$E' \rightarrow \varepsilon$	$E' \rightarrow \varepsilon$
T	$T \rightarrow FT'$					$T \rightarrow FT'$		
T'		$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$	$T' \rightarrow /FT'$		$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$
F	$F \rightarrow nb$					$F \rightarrow (E)$		

Les règles ajoutées en exécutant 1 sont en rouge

Les règles ajoutées en exécutant 2 sont en vert

2. Mise en œuvre d'un analyseur syntaxique

Analyse descendante déterministe

- Étant donné la table d'analyse LL, comment déterminer si un mot w appartient au langage généré par la grammaire?

→ **Algorithme d'analyse**

2. Mise en œuvre d'un analyseur syntaxique

Analyse descendante déterministe

Algorithme d'analyse

Étape 0: Choix du mot à analyser

Étape 1: Préparation de la table d'analyse M

Étape 2: Initialisation d'une pile

Étape 3: initialisation d'un pointeur **ps** qui pointe sur le premier caractère du mot

Algorithme

Soit **X** le symbole au sommet de la pile

Soit **a** le caractère pointé par **ps**

Si **X** est un non-terminal alors

Si $M[X,a] = X \rightarrow Y_1 \dots Y_n$ alors

Enlever **X** de la pile

Mettre Y_n puis Y_{n-1} puis... Y_1 dans la pile

Emettre en sortie la règle $X \rightarrow Y_1 \dots Y_n$

Sinon (case vide)

ERREUR

Fin si

Sinon

Si **X** = \$ alors

Si **a** = \$ alors

ACCEPTER

Sinon

ERREUR

Fin si

Sinon

Si **X** = **a** alors

Enlever **X** de la pile

Avancer **ps**

Sinon

ERREUR

Fin si

Fin si

Fin si

2. Mise en œuvre d'un analyseur syntaxique

Analyse descendante déterministe

- Sur l'exemple 3, analysez le mot $w = nb + nb * nb$

PILE	Entrée	Sortie
\$E	nb+nb*nb\$	$E \rightarrow TE'$
\$E'T	nb+nb*nb\$	$T \rightarrow FT'$
\$E'T'F	nb+nb*nb\$	$F \rightarrow nb$
\$E'T'(nb)	nb+nb*nb\$	
\$E'T'(+)	+nb*nb\$	$T' \rightarrow \epsilon$
\$E'(+)	+nb*nb\$	$E' \rightarrow +TE'$
\$E'T(+)	+nb*nb\$	
\$E'T	nb*nb\$	$T \rightarrow FT'$
\$E'T'F	nb*nb\$	$F \rightarrow nb$
\$E'T'(nb)	nb*nb\$	
\$E'T'(*)	*nb\$	$T' \rightarrow *FT'$

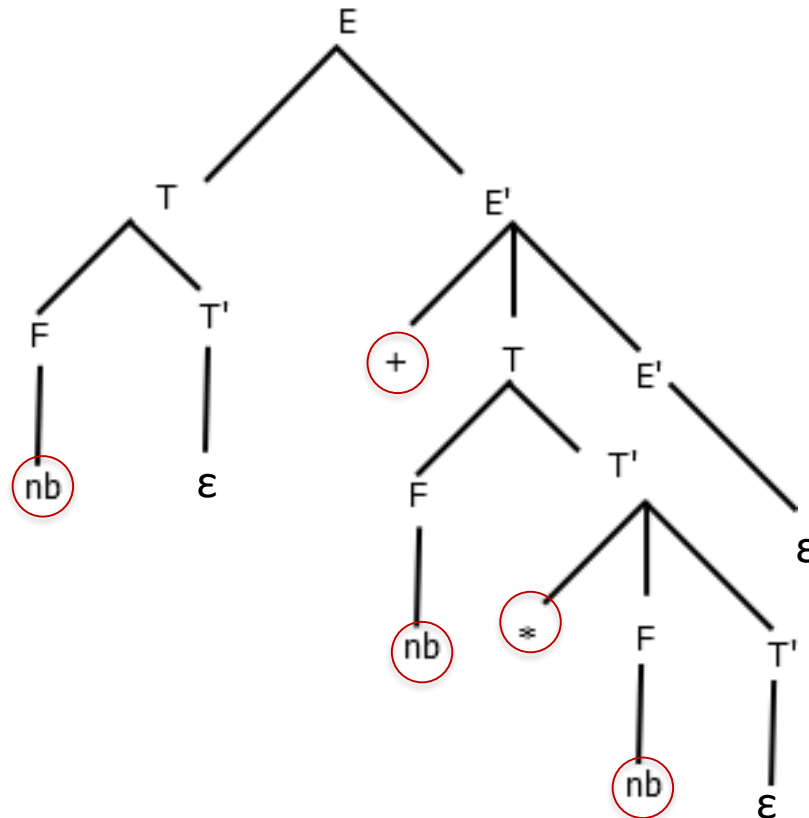
PILE	Entrée	Sortie
\$E'T'F(*)	*nb\$	
\$E'T'F	nb\$	$F \rightarrow nb$
\$E'T'(nb)	nb\$	
\$E'T'(\$)	\$	$T' \rightarrow \epsilon$
\$E'(\$)	\$	$E' \rightarrow \epsilon$
\$	\$	ACCEPTER

2. Mise en œuvre d'un analyseur syntaxique

Analyse descendante déterministe

- Sur l'exemple 3, analysez le mot **w**=nb+nb*nb

→ L'arbre syntaxique correspondante est donc :



2. Mise en œuvre d'un analyseur syntaxique

Analyse descendante déterministe

- Sur l'exemple 3, analyser le mot $w=(nb+nb)nb$

2. Mise en œuvre d'un analyseur syntaxique

Grammaire LL(1)

- On appelle grammaire LL(1) une grammaire pour laquelle la table d'analyse n'a aucune case définie de façon multiple.
- Exemple: Soit la grammaire suivante:
$$\begin{cases} S \rightarrow aAb \\ A \rightarrow cd|c \end{cases}$$

On a $\text{PREMIER}(A) = \{c\}$ et $\text{PREMIER}(S) = \{a\}$

On a $\text{SUIVANT}(A) = \{b\}$ et $\text{SUIVANT}(S) = \{\$ \}$

La table d'analyse de G est donc:

V_N	V_T	a	b	c	\$
S		$S \rightarrow aAb$			
A				$A \rightarrow cd$ $A \rightarrow c$	

→ Cette grammaire n'est pas LL(1)

4. Exercices

Exercice 1:

On considère la grammaire $G=(V_T, V_N, S, P)$ suivante où :

S désigne l'axiome de la grammaire

$V_T=\{a,b,c,d\}$ l'ensemble des symboles terminaux

$V_N=\{S,M\}$ l'ensemble des symboles non-terminaux

$P=\{S \rightarrow bSaa \mid M \mid \varepsilon$

$M \rightarrow cMc \mid d\}$

1. Construire les ensembles PREMIER et SUIVANT de cette grammaire
2. En déduire la table d'analyse LL correspondante.
3. Analyser le mot $\omega=bcdcaa$ et donner l'arbre de dérivation correspondante

4. Exercices

Exercice 2:

On se propose de modéliser un polynôme par la liste de ses coefficients. Par exemple $P(x)=3x^7-13x^5+9x-19$ sera modélisé par la liste (3 -13 9 -19) qui désigne les coefficients du polynôme. Pour générer un tel langage de listes, on utilise la grammaire $G=(V_T, V_N, S, P)$ suivante où :

S désigne l'axiome de la grammaire

$V_T=\{\text{nb}, (,)\}$ l'ensemble des symboles terminaux

$V_N=\{S, L, R\}$ l'ensemble des symboles non-terminaux

$P=\{S \rightarrow (L), L \rightarrow \text{nb } R, R \rightarrow L \mid \varepsilon\}$

1. Donner les ensembles PREMIER et SUIVANT pour les symboles non-terminaux de cette grammaire
2. Dresser la table d'analyse LL correspondante.
3. Donner l'arbre syntaxique pour le mot décrivant le polynôme $P(x)=3x^2-7x+1$.